

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	<i>Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)</i>		
Student Name:	G. Tejasree	Reg. No.:	192511204
Section / Batch:	Section A	Date:	30 August 2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

8. Motion Detection for Home Security

. Problem Understanding

The objective is to design an OpenCV-based home security system that continuously monitors a camera feed and detects significant movement. The system must:

- Detect motion from a live camera feed.
- Trigger an alert only when significant movement is detected.
- Reduce false alarms caused by camera noise, lighting changes, shadows, or small background movements.
- Operate continuously with reasonable processing speed.

A. Detect Motion from a Camera Feed

The first requirement is to continuously capture video from a security camera. OpenCV's `VideoCapture()` can be used to read frames from the webcam or CCTV camera.

Each captured frame is converted from BGR to grayscale because motion detection mainly depends on changes in intensity. A Gaussian Blur is then applied to reduce small camera noise.

The current frame is compared with the background using background subtraction such as MOG2. Areas that are different from the background are considered possible moving regions.

Process:

Camera → Frame Capture → Grayscale → Gaussian Blur → Background Subtraction

When a moving object enters the camera's view, the changed region appears as a foreground region.

B. Trigger Alerts Only for Significant Movement

Not every detected change should generate an alert. Small changes may be caused by noise, leaves, shadows, or minor lighting variations.

Therefore, the foreground image is processed using thresholding to obtain a binary image.

Morphological operations such as erosion and dilation can then remove small unwanted regions.

Next, OpenCV's `findContours()` is used to identify moving regions. The area of each contour is calculated using `contourArea()`.

A minimum area threshold is defined:

If contour area > minimum area:

 Significant motion detected

 Trigger alert

Else:

Ignore the movement

For example, a very small moving region can be ignored, while a person or other large moving object produces a larger contour and can be considered significant.

To make the system more reliable, the alert can be generated only when significant motion is present for several consecutive frames, rather than from a single frame.

C. Reduce False Alarms

False alarms are an important problem in a home security system. The system may incorrectly detect motion because of camera noise, shadows, lighting changes, or small background movements.

The proposed system reduces false alarms using the following methods:

1. Gaussian Blur – removes small image noise before motion analysis.
2. Background Subtraction – identifies regions that actually differ from the background.
3. Thresholding – removes very weak pixel changes.
4. Morphological Operations – removes small unwanted regions and improves the foreground mask.
5. Contour Area Filtering – ignores objects smaller than the selected minimum area.
6. Consecutive-Frame Confirmation – triggers an alert only when significant movement continues across multiple frames.
7. Background Updating – allows the system to adapt to gradual environmental changes such as lighting variations.

Complete OpenCV Pipeline

Camera Feed



Capture Frames



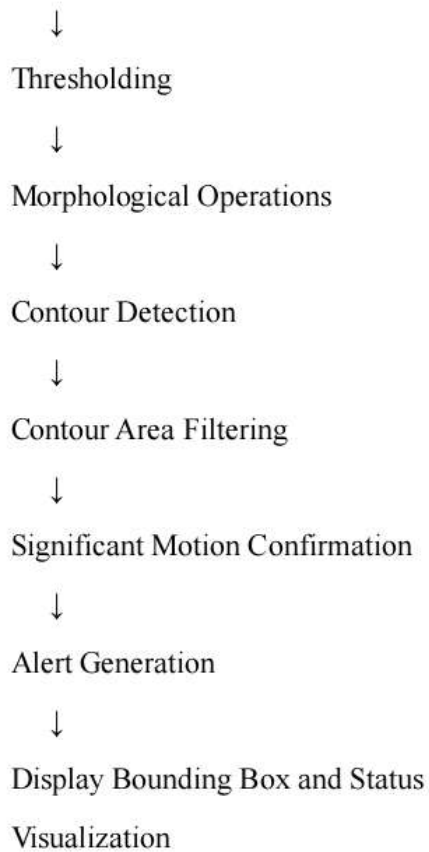
Convert to Grayscale



Gaussian Blur



Background Subtraction



When significant motion is detected, OpenCV drawing functions can be used:

- `rectangle()` → draws a bounding box around the moving region.
- `putText()` → displays "MOTION DETECTED".
- `imshow()` → displays the live processed video.

Justification

This approach is suitable for a home security system because it combines simple and efficient OpenCV operations. Grayscale conversion and Gaussian filtering reduce unnecessary processing, while background subtraction identifies motion. Thresholding, morphology, and contour-area filtering help remove noise and insignificant changes. Consecutive-frame confirmation further reduces false alarms. Therefore, the proposed pipeline can continuously monitor a camera feed and generate alerts mainly for meaningful movement.

IMPLEMENTATION

```
import cv2
# Open webcam
cap = cv2.VideoCapture(0)

# Background subtraction
background = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=50,
    detectShadows=True
)

motion_count = 0
required_frames = 5

while True:
    ret, frame = cap.read()

    if not ret:
        print("Camera not found")
        break

    # Resize frame
    frame = cv2.resize(frame, (640, 480))

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Remove noise
    blur = cv2.GaussianBlur(gray, (21, 21), 0)

    # Detect foreground
    mask = background.apply(blur)

    # Threshold
    _, mask = cv2.threshold(mask, 200, 255, cv2.THRESH_BINARY)

    # Remove noise using morphology
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.dilate(mask, kernel, iterations=2)
```

```

# Find moving objects
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

motion = False

for contour in contours:
    area = cv2.contourArea(contour)

    # Ignore small movements
    if area < 1500:
        continue

    motion = True

    # Draw bounding box
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(frame, (x, y), (x + w, y + h),
        (0, 255, 0), 2)

# Confirm significant motion
if motion:
    motion_count += 1
else:
    motion_count = 0

if motion_count >= required_frames:
    cv2.putText(frame, "MOTION DETECTED!",
        (20, 50), cv2.FONT_HERSHEY_SIMPLEX,
        1, (0, 0, 255), 3)
else:
    cv2.putText(frame, "NO SIGNIFICANT MOTION",
        (20, 50), cv2.FONT_HERSHEY_SIMPLEX,
        0.8, (255, 255, 255), 2)

# Display output
cv2.imshow("Home Security - Motion Detection", frame)
cv2.imshow("Motion Mask", mask)

# Press Q to exit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```
